

CODE GARDEN

A living, playable ecosystem for understanding and tending codebases

"For the first time, anyone can understand a codebase — not by reading it, but by watching it live, grow, and breathe."

Competition	OpenAI Build Week Challenge (Devpost)
Built With	Codex + GPT-5.6 (required)
Submission Deadline	Tuesday, July 21, 2026 - 5:00 PM PT
Judging Period	July 22 - August 5, 2026
Winners Announced	~August 12, 2026
Prize Pool	\$100,000 cash across tracks, plus OpenAI credits, DevDay passes, spotlight opportunities
Judging Criteria	Technical implementation - Design & UX - Potential impact - Quality of idea
Primary Track	Education (with deliberate crossover into all four tracks)

The Concept

Code Garden turns any codebase into a living garden that you can walk through, understand, and tend with your own hands. The state of the garden is determined by the actual state of the code — not a dashboard, not a chart. Functions are plants. Dependencies are root systems. Test coverage is sunlight. Bugs and vulnerabilities are pests. Dead code is decay. Technical debt is erosion.

You are the gardener. You hold real tools. When you clip a withered branch, Codex actually removes the dead code. When you water a drought zone, Codex actually writes the missing tests. The game layer isn't decoration — the interaction model forces understanding before action, which is what makes it a teaching tool as much as a developer tool.

The core promise: a non-coder — a student, a product manager, a founder — can walk through a codebase and genuinely understand it without reading a single line of code.

The Metaphor (Full Mapping)

Code Concept	Garden Element
Function / module	Individual plant
Dependencies / imports	Root systems connecting plants
Test coverage	Sunlight reaching the plant
Untested code	Drought zones (dry, brown patches)
Bugs / vulnerabilities	Pests, insects, rot
Dead / unused code	Withered branches, decay
Technical debt	Erosion, overgrowth
Bad dependency spreading	Invasive species creeping through roots
A refactor	Fertilizer - the plant grows stronger
Repo / team boundaries	Garden plots in a shared landscape
Sprint cycles	Seasonal cycles
Git history	The passage of seasons over the garden's life

The Gardener's Tools (Core Gameplay)

Tool	Player Action	What Codex Actually Does
Clippers	Snip dead branches and withered growth	Identifies dead/unused code, explains what it is, and removes it on confirmation
Watering Can	Pour water on dry, brown drought zones	Writes missing tests for untested code; the patch visibly greens up in real time
Pesticide Spray	Spray insects and rot	Patches security vulnerabilities and bugs
Fertilizer	Sprinkle on a struggling plant	Refactors the function; the plant grows visibly taller and stronger
Magnifying Glass	Hover over any plant	Plain-English explanation of what that code does - the non-coder superpower

Design rule: you cannot act without understanding. You can't spray pesticide without seeing the bug. You can't clip without Codex explaining what you're cutting. The game teaches through doing.

Signature Features (The Originality Layer)

1. The Garden Has Memory — Time as Seasons

Scrub through git history like scrubbing through time. Watch the garden spring to life when the repo was young, grow wild during a growth sprint, wither during a bug crisis, and bloom after a refactor. Git history reframed from a list of commits into something felt. Nobody has done this.

2. Plants Have Personality

Each plant's character is derived from its commit history. Code written at 2 AM under deadline pressure looks scraggly and stressed. Code crafted carefully over weeks stands tall and well-rooted. The human story behind the code becomes visible. Codex narrates like a naturalist: "This one's been here since the beginning. Nobody touches it. Everyone's afraid of it."

3. The Garden Talks Back

Hover over a plant and it speaks for itself — not a tooltip, a voice. The function explains what it does, what it needs, what it's afraid of. Dead code pleads its case before you clip it. A bug caught mid-infestation panics. This is the non-coder magic moment: the code introduces itself.

4. Invasive Species Mechanic

A bad dependency gets introduced and you watch it spread through the garden in real time — roots creeping under other plants, crowding them out, changing their color. Cascade failures become visible before they happen. Rip it out and watch the garden recover. Dependency risk made visceral.

5. AI-Painted Gardens (Codex Image Generation)

Codex generates the garden's visuals. Every repo gets a genuinely unique, AI-generated landscape reflecting its actual health — a beautiful codebase looks like a lush oil painting; a legacy nightmare looks like a dying swamp. Generated, not templated.

6. The Garden Tends Itself Overnight

Using Codex's background/parallel task execution, the garden keeps working while you're away. You return in the morning to a caretaker's changelog: "Watered three drought zones, clipped two dead branches, spotted a new infestation in the auth module." The garden is alive even when you're not there.

7. Persistent Memory — The Garden Ages

Codex's cross-session memory means the garden accumulates lived history. It remembers when each area was last tended, which pests keep coming back, which plants never thrive. The garden isn't reset every session — it ages.

8. Custom Skills — Teach the Garden Your Rules

Codex Skills (shareable, checked into the repo) let teams encode their culture: "Never clip anything without two reviewers." "Always water tests before trimming." The garden learns your rules, not just generic best practices.

9. Visual App Inspection (Codex Computer Use)

Codex can open the running app in its in-app browser, look at it visually, and map what it sees back to the garden. A broken UI component shows as a withered flower because Codex actually saw it was broken — visual inspection, not just static analysis.

How It Hits All Four Challenge Tracks

Developer Tools (the engine)

The heart of the product. Codex analyzes the repo in real time — decay, overgrown dependencies, vulnerabilities, drought zones — and doesn't just flag issues, it acts: opens PRs, proposes refactors, and explains its reasoning. The garden is the interface for a genuinely powerful agentic coding tool.

Education (the primary track)

The garden metaphor is arguably the most intuitive explanation of software architecture ever. A teacher shows a class a healthy codebase — lush, well-rooted, full of sunlight — next to a legacy nightmare full of invasive species and root rot. Students grasp modularity, coupling, coverage, and debt visually before reading a line of code. Codex narrates like a naturalist. Strategic note: one Build Week judge is OpenAI's VP of Education, and this track is likely the least crowded.

Work & Productivity (the team view)

Zoom out from one repo to an organization. Each team's codebase is a plot in a shared landscape. Which plots are thriving, which are overgrown, which are dying — visible at a glance. Ownership is visible. Sprint health maps to seasons. Tech debt appears as erosion.

Apps for Your Life (the personal layer)

Your GitHub profile becomes a personal garden that grows as you commit and changes with the seasons of your career. Meditative and reflective — your coding life as something living. Identity, not just utility.

Audiences

- **Non-coders first:** students, product managers, founders, investors, stakeholders — people who use software but never see the code. Code transparency for them is almost completely unsolved.
- **Developers:** a genuinely useful agentic maintenance tool wearing a delightful interface.
- **Teachers & students:** the classroom mode — compare healthy vs. sick codebases to teach architecture concepts visually.
- **Teams:** the org-landscape view for engineering health at a glance.

Originality & Competitive Landscape

The "codebase as city" metaphor is well-established (CodeCharta's 3D city maps; Code Park's walkable 3D environment). Gamified coding-education tools (CodeCombat, CodinGame) teach writing code, not understanding existing codebases. An abandoned repo literally named `code_garden` exists with 1 star — nobody ran with the idea.

What doesn't exist anywhere:

- The garden metaphor executed seriously (the city metaphor is owned; the garden is wide open)
- Interactive tools that trigger real AI actions — existing visualizers are read-only; Code Garden acts
- A non-coder-first audience — everything out there is built for developers reading their own code
- An AI (Codex) as the autonomous intelligence layer tending the garden
- Time-scrubbing git history as seasons; plants with commit-derived personality; a garden that talks

The city metaphor being taken works in our favor — judges will immediately see the contrast.

The Demo Video (<3 Minutes, Required)

Open a famously messy open-source repo. The garden is overgrown, brown, crawling with pests. Spend 90 seconds tending it — clipping, spraying, watering. Cut to: the same garden, lush and healthy. Then show the actual code changes that happened underneath (real PRs, real tests, real diffs). Per the rules, audio must explicitly cover how Codex AND GPT-5.6 were used, and the submission should highlight where Codex accelerated the workflow and where key decisions were made.

Submission Requirements Checklist

- A working project built with Codex using GPT-5.6
- A chosen category/track
- A project description: what it is and how it works
- A public YouTube demo video under 3 minutes, with audio covering how Codex and GPT-5.6 were used
- A code repository link — if private, shared with testing@devpost.com and build-week-event@openai.com; if public, with an open source license
- A README with setup instructions, sample data if needed, and clear steps to run the project
- Documentation of architecture and key decisions

Build Priorities (4-Day Reality Check)

Must Have (the demo depends on these)

- Repo analysis pipeline: parse a codebase, compute health signals (dead code, test coverage, complexity, vulnerabilities)
- The garden rendering: repo state mapped to a visual garden that clearly reads as healthy vs. sick
- At least 2-3 working tools end-to-end (Clippers + Watering Can + Magnifying Glass recommended) where the action triggers a real Codex change
- The plain-English Magnifying Glass explanations — the education/non-coder core
- Before/after payoff moment for the demo video

Should Have (differentiators, if time allows)

- Git-history season scrubbing (even a simplified 3-4 snapshot version)
- Plants talking back (text-based voice is fine; audio is a bonus)
- Codex-generated garden artwork per repo

Nice to Have (mention in the writeup as roadmap)

- Overnight autonomous tending with morning changelog
- Org-level multi-plot landscape view
- Invasive species spread animation
- Custom Skills for team rules
- Computer-use visual UI inspection
- Personal GitHub-profile gardens

Why It Wins

- **Quality of idea:** a genuinely unclaimed metaphor with an emotional hook — code as something alive
- **Technical implementation:** real agentic Codex actions (PRs, tests, refactors), not a wrapper UI
- **Design & UX:** the game layer is the interface; understanding is required before action
- **Potential impact:** opens codebases to the millions of people who can't read them — students, PMs, founders, stakeholders
- **Track strategy:** one coherent product that naturally serves all four tracks, anchored in the least-crowded one (Education) with a judge who leads Education at OpenAI

"This is the algorithm that shows you content. Here's what's healthy. Here's what's rotting. Here's what the team has been neglecting." — Code transparency for everyone.